

Rapport de Projet - DevOps Calculator App

Prepared By : KHADIJA IBISK

1. Introduction

Le projet consiste à développer une application Full Stack que nommée « DevOps Calculator App », composée :

- d'un frontend React,
- d'un backend Node.js,
- d'une API REST,
- d'une containerisation Docker,
- d'une gestion de version Git/GitLab,
- ainsi qu'une initiation au CI/CD avec Jenkins.

Ce projet permet de mettre en œuvre une architecture moderne utilisée dans les environnements professionnels.

2. Présentation du projet

Le projet « DevOps Calculator App » est une application web simple permettant d'effectuer des opérations mathématiques :

- Addition
- Soustraction
- Multiplication
- Division

L'utilisateur saisit deux nombres dans le frontend React.

Le frontend envoie ensuite une requête HTTP vers le backend Node.js via une API REST.

Le backend effectue le calcul puis renvoie le résultat au frontend.

3. Technologies utilisées

Technologie	Rôle
React.js	Frontend
Node.js	Backend
Express.js	Serveur API
Axios	Communication HTTP
Docker	Containerisation
Docker Compose	Orchestration
Git	Gestion de version
GitLab	Hébergement du code
Jenkins	CI/CD

VS Code	IDE de développement

4. Architecture du projet

Architecture générale

Frontend React
↓
Axios HTTP Requests
↓
Backend Node.js API
↓
Docker Containers
↓
GitLab Repository
↓
CI/CD Jenkins

5. Structure du projet

```
/devops-project
│
├── backend
│   ├── server.js
│   ├── package.json
│   └── Dockerfile
│
├── frontend
│   ├── src
│   ├── public
│   ├── package.json
│   └── Dockerfile
│
├── docker-compose.yml
├── .gitlab-ci.yml
└── README.md
```

6. Mise en place de l’environnement

Installation des outils

Les outils suivants ont été installés :

- Node.js
- npm
- Docker Desktop
- Git
- VS Code
- Jenkins

Vérification des installations

Vérification Node.js

```
node -v
```

Vérification npm

```
npm -v
```

Vérification Docker

```
docker --version
```

Vérification Git

```
git --version
```

7. Développement du Backend

Création du backend

Le backend a été développé avec Express.js.

Installation des dépendances

npm init -y
npm install express cors

Fonctionnalités API

Le backend contient les routes suivantes :

Route	Méthode	Fonction
/add	POST	Addition
/subtract	POST	Soustraction
/multiply	POST	Multiplication
/divide	POST	Division

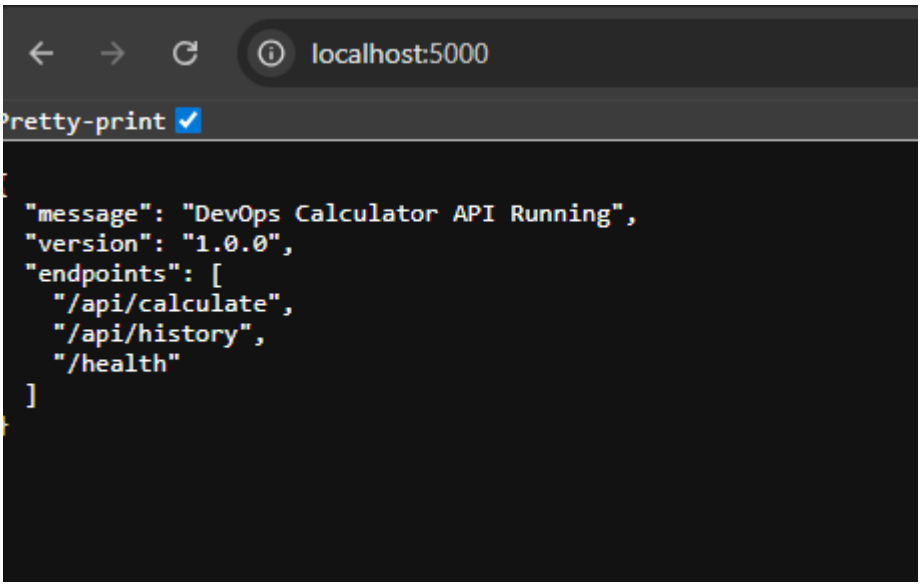
Lancement du serveur

```
node server.js
```

Résultat

Le serveur fonctionne sur le port :

http://localhost:5000



8. Développement du Frontend

Création du frontend React

Le frontend a été créé avec Create React App.

Création du projet

```
npx create-react-app frontend
```

Installation Axios

```
npm install axios
```

Fonctionnement

L'utilisateur :

1. saisit deux nombres,
2. clique sur une opération,
3. le frontend envoie la requête,
4. le résultat s'affiche.

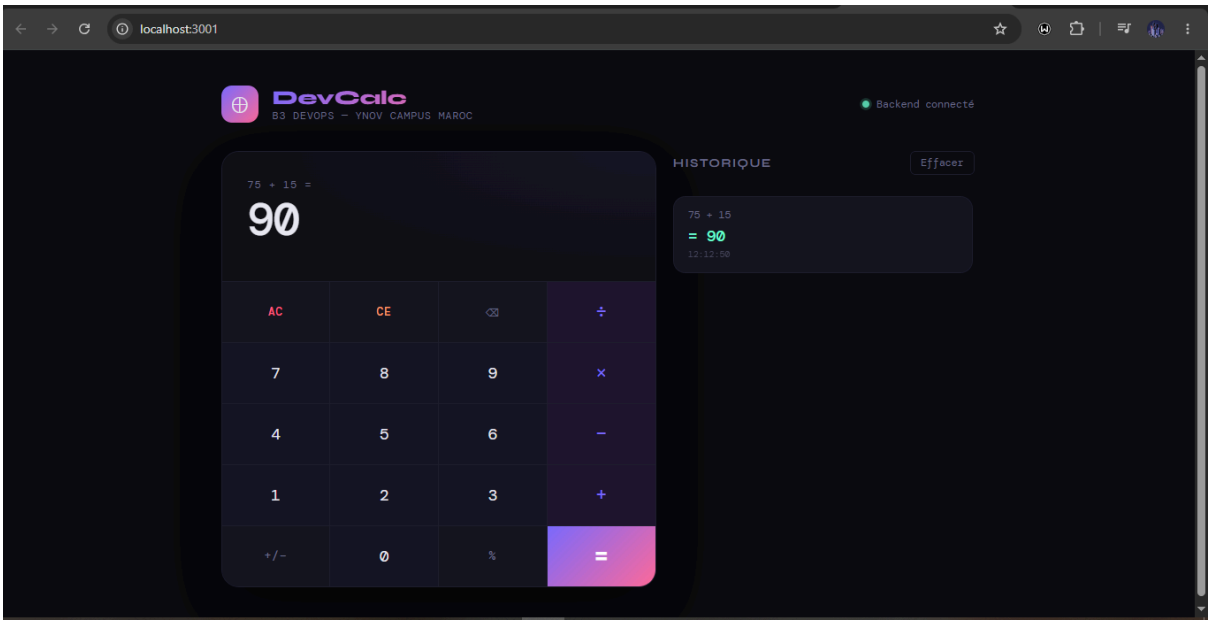
Lancement du frontend

```
npm start
```

Résultat

Le frontend fonctionne sur :

http://localhost:3000



9. Communication Frontend / Backend

La communication est réalisée via des requêtes HTTP.

Le frontend React utilise Axios pour envoyer les données.

Le backend Express traite les calculs et renvoie une réponse JSON.

10. Dockerisation du projet

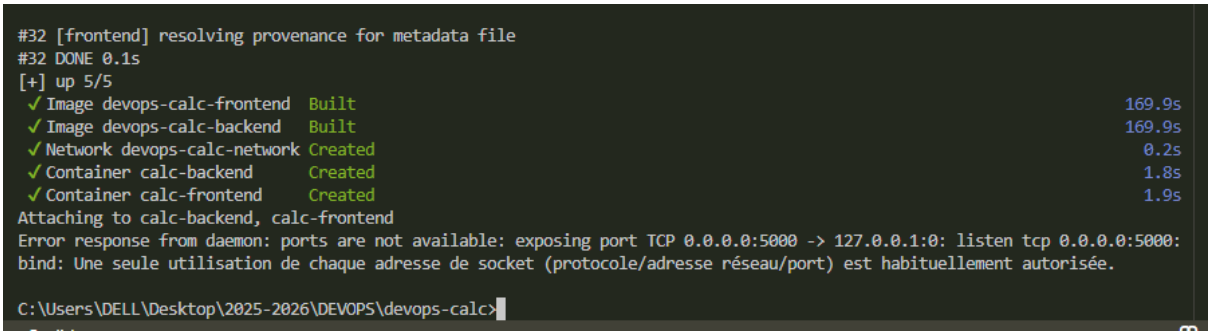
Objectif

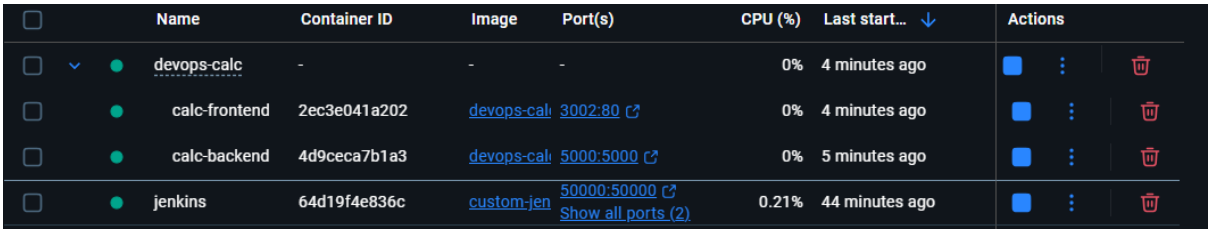
Docker permet d'exécuter l'application dans des containers afin d'assurer :

- la portabilité,
- la reproductibilité,
- l'isolation des services.

Lancement des containers

```
docker compose up --build
```





11. Utilisation de Git et GitLab

Initialisation Git

```
git init
```

Ajout des fichiers

```
git add .
```

Commit

```
git commit -m "Initial project setup"
```

Push GitLab

```
git push origin main
```

- sauvegarder le code,
- suivre les modifications,
- collaborer,
- connecter le projet au CI/CD.

12. CI/CD et Jenkins

Objectif

Automatiser certaines étapes du développement :

- récupération du code,
- build,
- exécution des containers.

Jenkins

Jenkins a été installé dans un container Docker.

Pipeline Jenkins

```
pipeline {

      agent any

      stages {
```

```
        stage('Clone Repository') {

                steps {

                        git branch: 'main',
                           credentialsId: 'gitlab-token',
                           url: 'https://gitlab.com/...'

                }
        }

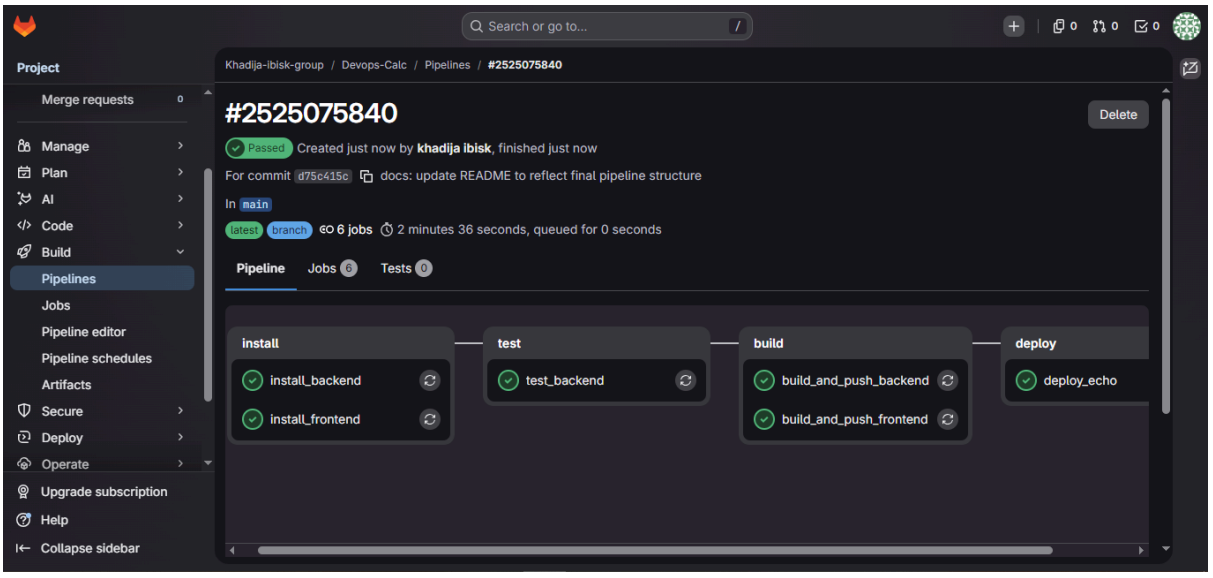
        stage('Build Docker Images') {

                steps {

                        sh 'docker compose build'

                }
        }

        }
}
```



Difficultés rencontrées

Certaines difficultés ont été rencontrées avec :

- les permissions Docker sous Windows,
- l'accès Docker depuis Jenkins,
- les volumes Docker.

Malgré cela, l’architecture CI/CD a été mise en place et comprise.

13. Tests réalisés

Tests Backend

Tests des routes API :

- /add
- /subtract
- /multiply
- /divide

Tests Frontend

- affichage des composants,
- communication avec API,
- affichage des résultats.

Tests Docker

- build des images,
- lancement des containers,
- mapping des ports.

14. Difficultés rencontrées

Plusieurs difficultés techniques ont été rencontrées :

Docker

- conflits de ports,
- containers déjà actifs,
- gestion des volumes.

Jenkins

- permissions Docker,
- accès au socket Docker,
- authentification GitLab.

Frontend / Backend

- erreurs Cannot GET /,
- gestion des routes,
- communication API.

Ces problèmes ont permis d’approfondir la compréhension des environnements DevOps.

15. Résultats obtenus

Le projet final permet ceci et tout ceci marche correctement:

une architecture Full Stack,

un frontend React,

un backend Node.js,

une API REST,

une communication HTTP,

une containerisation Docker,

l’utilisation de GitLab,

une introduction au CI/CD.

16. Compétences acquises

Au cours du projet, plusieurs compétences ont été développées.

Développement

- React.js
- Node.js
- Express.js
- API REST
- Axios

DevOps

- Docker
- Docker Compose
- Git
- GitLab
- Jenkins
- CI/CD

Méthodologie

- séparation frontend/backend,
- architecture moderne,
- debugging,
- automatisation.

17. Améliorations possibles

Le projet peut être amélioré avec :

- une base de données MongoDB,
- un système d'authentification,
- un historique des calculs,
- un design responsive,
- un mode sombre,
- un déploiement Kubernetes,
- un hébergement cloud.

18. Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux du développement Full Stack et du DevOps.

L'application « DevOps Calculator App » constitue une base solide permettant de comprendre :

- la séparation frontend/backend,
- les API REST,
- Docker,
- GitLab,
- Jenkins,
- le CI/CD.

Malgré certaines difficultés techniques, le projet a permis d'acquérir une expérience concrète proche des environnements professionnels.

19. Annexes

Commandes principales

Backend

```
cd backend
 node server.js
```

Frontend

```
cd frontend
 npm start
```

Docker

```
docker compose up --build
```

Git

```
git add .
 git commit -m "update"
 git push
```